

Microservices, Eureka Discovery and API Gateway

1. Account Microservice

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

AccountController.java

```
package com.cognizant.account.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Map<String, Object> getAccount(@PathVariable String number) {
        Map<String, Object> account = new HashMap<>();
        account.put("number", number);
        account.put("type", "savings");
        account.put("balance", 234343);
        return account;
    }
}
```

Request

```
curl -s http://localhost:8080/accounts/00987987973432
```

Output

```
{"number":"00987987973432","type":"savings","balance":234343}
```

2. Loan Microservice

application.properties

```
server.port=8081
```

LoanController.java

```

package com.cognizant.loan.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class LoanController {

    @GetMapping("/loans/{number}")
    public Map<String, Object> getLoan(@PathVariable String number) {
        Map<String, Object> loan = new HashMap<>();
        loan.put("number", number);
        loan.put("type", "car");
        loan.put("loan", 400000);
        loan.put("emi", 3258);
        loan.put("tenure", 18);
        return loan;
    }
}

```

Request

```
curl -s http://localhost:8081/loans/H00987987972342
```

Output

```
{"number":"H00987987972342","type":"car","loan":400000,"emi":3258,"tenure":18}
```

Running the loan service on the default port 8080 while the account service is already using it fails with "Web server failed to start. Port 8080 was already in use." Setting server.port=8081 resolves this.

3. Eureka Discovery Server

pom.xml

```

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

```

application.properties

```

server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
logging.level.com.netflix.eureka=OFF
logging.level.com.netflix.discovery=OFF

```

EurekaDiscoveryServerApplication.java

```

package com.cognizant.eurekadiscoveryserver;

import org.springframework.boot.SpringApplication;

```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaDiscoveryServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaDiscoveryServerApplication.class, args);
    }
}
```

Request

```
curl -s http://localhost:8761/eureka/apps
```

Output

```
Instances currently registered with Eureka: (none)
```

4. Registering Account and Loan Services with Eureka

pom.xml – additions to account and loan projects

```
<properties>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
</properties>

<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

AccountApplication.java

```
package com.cognizant.account;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;

@SpringBootApplication
@EnableDiscoveryClient
public class AccountApplication {
```

```

    public static void main(String[] args) {
        SpringApplication.run(AccountApplication.class, args);
    }
}

```

application.properties – account service

```

spring.application.name=account-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

```

application.properties – loan service

```

server.port=8081
spring.application.name=loan-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

```

Request – after starting eureka-discovery-server, account and loan

```

curl -s http://localhost:8761/eureka/apps

```

Output

```

Instances currently registered with Eureka:
ACCOUNT-SERVICE    n/a (1)    UP (1) - host.docker.internal:account-service:8080
LOAN-SERVICE       n/a (1)    UP (1) - host.docker.internal:loan-service:8081

```

5. Greet Microservice

pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

```

application.properties

```

spring.application.name=greet-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

```

GreetController.java

```

package com.cts.greet.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetController {

    @GetMapping("/greet")
    public String sayHello() {
        return "Hello World!!";
    }
}

```

```
}
```

Request

```
curl -s http://localhost:8080/greet
```

Output

```
Hello World!!
```

6. API Gateway with Global Logging Filter

pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

application.properties

```
server.port=9090
spring.application.name=api-gateway
spring.cloud.gateway.discovery.locator.enabled=true
spring.cloud.gateway.discovery.locator.lower-case-service-id=true
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

LogFilter.java

```
package com.cts.gateway.filters;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;

import reactor.core.publisher.Mono;

@Component
public class LogFilter implements GlobalFilter {

    Logger logger = LoggerFactory.getLogger(LogFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        logger.info("====>Request URL {}", exchange.getRequest().getURI());
        return chain.filter(exchange);
    }
}
```

```
}  
}
```

ApiGatewayApplication.java

```
package com.cts.gateway;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
public class ApiGatewayApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(ApiGatewayApplication.class, args);  
    }  
}
```

Request – Eureka registry after starting greet-service and api-gateway

```
curl -s http://localhost:8761/eureka/apps
```

Output

```
Instances currently registered with Eureka:  
API-GATEWAY    n/a (1)  UP (1) - host.docker.internal:api-gateway:9090  
GREET-SERVICE n/a (1)  UP (1) - host.docker.internal:greet-service:8080
```

Request – call greet-service through the gateway

```
curl -s http://localhost:9090/greet-service/greet
```

Output

```
Hello World!!
```

Console log – api-gateway

```
INFO 14328 --- [ctor-http-nio-2] com.cts.gateway.filters.LogFilter      : ==>Request  
URL http://localhost:9090/greet-service/greet
```